

address this issue. Perhaps most novel in its approach was Apple Inc, when it created its Time Machine. This had no user-configurable options because there didn't need to be any. All Time Machine ever did was incrementally back up all of the hard disk to a Time Capsule (which was nothing but a NAS machine, or for practical purposes, a portable hard drive attached using the network instead of the USB). You could then click a button on the file manager and be shown a 3D screen where you would be able to recover the file from any previous date. Say a document was created on Day 1, 25 per cent of it was completed on Day 2, 50 per cent on Day 3, 15 per cent on Day 4 and it got corrupted on Day 5. Using Time Machine, you could then see the file as it existed on Days 1 through 4, and then recover any version of the file you desired, such as the 25 per cent from Day 2 or the 90 per cent from Day 4. It still exists, and is one of the many crowning features of the Mac OS.

However, it's the open world that offers the best solution available. OpenSolaris uses the ZFS file system, and snapshots are built in as a feature. On OpenSolaris, it is just a matter of opening up Nautilus, right clicking the file, selecting the appropriate option and moving the Time Slider to the time you want. It doesn't get better than this.

Sadly, Linux doesn't get to use ZFS because of licensing restrictions, so we have to settle for Time Machine-like software. Presenting 'Back In Time', the king of automated snapshots and back-ups, in Linux!

Time travel, anyone?

There have been many such software, most famously, Time Vault and FlyBack, but none have ever got the equation right. Either the software is too complex to use, or it just doesn't work well. We've been neglecting the UNIX philosophy here, which is to do one task and do it well! But with *Back In Time*, the *attayachar* [harassment] is over.

Did I sound philosophical or boring? Well, back-ups are important and good back-up software are hard to come by. So let's get on with the software.

Back In Time is basically a pretty GUI tucked on top of the following command-line utilities: *cp*, *rsync* and *ln*. Yes, it basically copies, links and syncs files—a very simple idea, but very effective.

For the first back-up, or snapshot, all the specified data to back up is backed up into a unique time-named subdirectory of a master directory that you specify. For every subsequent back-up, *rsync* only copies the changed files to a new time-named directory, and *ln* creates links to the non-changed files from the previous snapshot directory. Pretty, right?

It also means that you can browse your back-ups like a normal filesystem and don't need BIT (*Back In Time*) or third-party software to restore a back-up if your OS is trashed: you can manually do so by simply copying

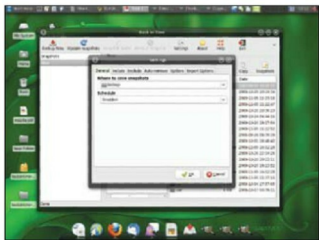


Figure 1: Back In Time user interface

the files, if need be. It also means that because the snapshots are not compressed, the back-ups are larger than the size of your source. Sure, that's a negative point, but most of us keep our back-ups on very large portable hard disks, anyway.

BIT doesn't even have a daemon of its own. All the actual back-ups are done by—wait for it—*cron*! Yes, that's it! Rather than reinvent the wheel, where you are scheduling an *rsync* job, all you do is add a line to the *crontab* and you are good to go!

Installing it

It's too simple for words if you're using Ubuntu, Fedora, Mandriva, or openSUSE. It's already in the Ubuntu, Fedora and Mandriva repos, so search for *backintime-common* and *backintime-gnome* for GTK and *backintime-kde* for KDE4.1. In openSUSE, you'll get it in the third-party packman repo, and in Ubuntu use the default package manager of your distro to install the package(s).

I've been using Slackware, and it's one of the few distros that need compilation from source code (I guess openSUSE has a build-service repo too). Added to that, I've been using Xfce4 rather than KDE4, and we have a unique installation scenario here.

The first run

You should only need to run this program once. Set it up, schedule the snapshotting interval, and then remove it. Yes, remove it. Because once the scheduling is complete, you don't need BIT any more, unless you want to change the snapshotting settings, or need a pretty GUI to recover your files. Anyway, the first time you run it (be sure to run it as the root), the settings dialogue shows up (Figure 1).

The first tab is the *General* tab. It lets you select a place to keep your snapshots, and lets you specify a scheduling interval. I'd say the one-hour scheduling interval is just right: enough snapshots and not too many unnecessary back-ups. Interestingly, there's a five-